

ResuMatch ATS Studio

System Architecture, Vector Embeddings & Technical Specifications

Stack: Python 3.14 (FastAPI) + React JS (Vite)	Version: 1.2.0 (Production Architecture)
Vector Dimension: 128-D Dense Latent Semantic Embeddings	Document Scope: End-to-End System Specifications

1. Executive Summary & Project Purpose

ResuMatch ATS Studio is an intelligent, full-stack recruitment engineering platform designed to bridge the gap between job candidates and modern Applicant Tracking Systems (ATS) like Workday, Taleo, Greenhouse, and Lever. Modern ATS platforms utilize multi-stage screening filters combining keyword taxonomies, TF-IDF lexical frequency, formatting compliance, and high-dimensional semantic vector embeddings to rank candidates.

The application accepts candidate resumes via **direct file upload (PDF, DOCX, TXT)** with no manual typing needed, ingests target job descriptions, computes an empirical **ATS Match Score (0–100%)** with point-by-point vector resonance, and automatically synthesizes an updated, tailored resume featuring STAR-method (Situation, Task, Action, Result) achievements and downloadable Microsoft Word (.docx) export.

2. High-Level System Architecture

The system follows a modular, decoupled client-server architecture. The backend is built with **Python 3.14 and FastAPI** providing high-performance asynchronous REST endpoints, while the frontend is built with **React JS and Vite** utilizing a modern dark glassmorphic design system.

Module	Technology / Libraries	Key Functional Responsibilities
Frontend SPA	React 18, Vite, Vanilla CSS, Lucide Icons	Upload-only file dropzone, radial SVG score gauge, interactive Keyword Gap Matrix, Vector Resonance table, and side-by-side diff viewer.
API Gateway	FastAPI, Uvicorn, CORS, StreamingResponse	Asynchronous request orchestration, multipart file ingestion, validation, and docx binary streaming.
Document Parser	pypdf, python-docx, Regex	Text extraction from binary PDF/DOCX, section segmentation (Experience, Education, Skills), contact metadata extraction.
Vector Engine	Scikit-Learn, TruncatedSVD, Cosine Similarity	128-D dense vector embeddings generation, overall vector cosine distance, point-by-point requirement matching.
ATS Scoring Engine	Multi-factor weighted algorithm, TF-IDF	Multi-factor scoring (Hard Skills, Vector, TF-IDF, Readability, Soft Skills, Experience), letter grade assignment.
Resume Optimizer	NLP rewrite rules, STAR templates	Dynamic tailored summary generation, STAR-method bullet rewriting with injected missing keywords, zero static data.
Document Exporter	python-docx, XML Open Packaging	Generates ATS-standard Microsoft Word documents with compliant typography, margins, and native bullet structures.

3. Vector Embeddings & Semantic Vectorization Engine

Modern recruiting platforms employ dense vector embeddings to measure contextual proximity rather than purely relying on exact string matches. The **VectorEmbeddingEngine** (implemented in `vector_engine.py`) provides continuous 128-dimensional dense vector embeddings:

- **Dual-Ngram Latent Semantic Projection:** Ingests text and builds unigram and bigram term distributions with sublinear term-frequency scaling to prevent outlier token domination.
- **Dimensionality Reduction (TruncatedSVD):** Reduces the high-dimensional sparse token matrix into a compact 128-dimensional dense continuous latent semantic space (preserving over 90% of semantic variance).
- **L2 Vector Normalization:** Normalizes vectors to unit length so that Cosine Similarity equals the inner dot product:
 $\cos(\theta) = (u \cdot v) / (||u|| \ ||v||)$.
- **Point-by-Point Requirement Resonance:** The engine splits the job description into discrete responsibility and qualification statements, vectorizes each item individually, and matches it against each bullet vector in the candidate's resume.
- **Coverage Metrics:** Produces a coverage percentage showing what proportion of JD requirement vectors have a high resonance ($\geq 72\%$) or moderate resonance (52–71%) in the candidate's resume.

4. Multi-Factor ATS Compatibility Scoring Algorithm

To simulate real enterprise ATS scanners, the platform does not rely on a single metric. Instead, it computes an empirical weighted composite score:

Scoring Dimension	Weight	Evaluation Methodology & Logic
Hard Technical Skills	30%	Exact and synonym matching against 1,000+ categorized skills (languages, frameworks, databases, cloud, DevOps, AI/ML tools).
Dense Vector Embeddings	20%	128-D dense vector cosine similarity measuring overall contextual resonance and semantic alignment between candidate profile and JD.
TF-IDF Lexical Similarity	15%	Evaluates broader vocabulary overlap, domain terminology density, and technical jargon consistency.
ATS Structural Readability	15%	Structural scan: contact info (email, phone, LinkedIn), standard section headers (Experience, Skills, Education), bullet point density (≥ 5), optimal word count (400-1000).
Soft Skills & Leadership	10%	Presence of essential execution keywords: cross-functional collaboration, mentorship, agile delivery, problem-solving, stakeholder management.
Experience & Seniority	10%	Detection of years of experience stated in candidate resume versus years required by JD (e.g. 4+ years), plus degree matching.

5. Dynamic Resume Tailoring & Optimization Pipeline

When the user clicks '**Update Resume for JD**', the system invokes `optimize_resume()` in `resume_optimizer.py`. The tailoring pipeline operates with **Zero Static Fake Data**, adhering strictly to candidate authenticity:

1. **Keyword Gap Extraction:** Identifies skills present in the target job description that are missing from the candidate's resume.

- 2. **Dynamic Summary Synthesis:** Crafts a targeted Professional Summary mentioning the exact target role and candidate's actual extracted years of experience (without hardcoding fake numbers).
- 3. **STAR Bullet Point Optimization:** Upgrades existing work experience bullet points using the **Situation, Task, Action, Result** framework, naturally weaving in missing technical tools and quantified impact metrics.
- 4. **Skills Matrix Restructuring:** Organizes skills into standard ATS categories (Programming Languages, Frameworks, Cloud & DevOps, Databases) with missing competencies injected.
- 5. **Candidate Profile Preservation:** Extracts the candidate's real name and header directly from their document, preserving authentic education and credentials.

6. Frontend UI/UX Architecture & Workflow

The React single-page application features a streamlined, high-aesthetic layout:

- **Centered Branding Header:** Glowing ResuMatch ATS logo and title centered at the top.
- **Upload-Only Candidate Resume:** Drag-and-drop file upload with format badges (.PDF, .DOCX, .TXT). Manual typing is disabled on the candidate resume panel to ensure ATS parsing fidelity.
- **Target Job Description Input:** Direct textarea allowing recruiters or candidates to paste JD requirements.
- **Vector Resonance & Keyword Matrix:** Shows point-by-point requirement matches with vector similarity percentages and color-coded status pills (green, amber, rose).
- **Direct Export Actions:** One-click Word document download (.docx), clipboard copy, and print-to-PDF without needing an intermediary manual editor.

7. REST API Endpoints Specification

Endpoint	Method	Payload / Parameters	Response
/api/parse-resume	POST	Multipart form file (PDF, DOCX, TXT)	JSON with extracted text, contact metadata, detected sections.
/api/analyze-ats	POST	JSON: { resume_text, job_description }	JSON with overall ATS score, vector analysis, keyword breakdown, checklist.
/api/optimize-resume	POST	JSON: { resume_text, job_description }	JSON with tailored summary, STAR bullets, updated resume, projected score.
/api/export-docx	POST	JSON: { resume_text, filename }	Binary attachment stream of ATS-formatted .docx document.
/api/health	GET	None	JSON health status: { status: 'healthy' }